

Техническая спецификация интеграции с Контур.КЭДО

Дата актуализации: 2026-05-18.

Документ описывает текущий алгоритм работы приложения `kontur` с Контур.КЭДО: какие endpoint приложения используются, какие методы КЭДО вызываются, какие параметры передаются и какие данные возвращаются.

1. Назначение

Приложение является промежуточным web-сервисом между пользователем и API Контур.КЭДО.

Основные функции:

- получить доступные типы документов КЭДО постранично по 20 штук;
- получить сотрудников организации;
- получить доступные типы подписи из конфигурации приложения;
- загрузить файл во временное хранилище КЭДО;
- обработать файл под выбранный тип документа;
- создать процесс подписания документа;
- проверить подписанные документы за период;
- скачать исходный content или печатную форму документа;
- выполнить диагностические проверки.

2. Конфигурация

Переменные окружения читаются через префикс `KONTUR_`.

Переменная	Назначение
<code>KONTUR_KEDO_BASE_URL</code>	Базовый URL API КЭДО. Для тестового стенда: <code>https://api.testkontur.ru/kedo</code> .
<code>KONTUR_KEDO_API_KEY</code>	API-ключ КЭДО. Если не задан, используется <code>KONTUR_CLIENT_SECRET</code> , затем <code>KONTUR_API_KEY</code> .
<code>KONTUR_KEDO_LOGIN</code> или <code>KONTUR_LOGIN</code>	Логин для password-аутентификации КЭДО.
<code>KONTUR_KEDO_PASSWORD</code> или <code>KONTUR_PASSWORD</code>	Пароль для password-аутентификации КЭДО.
<code>KONTUR_KEDO_ORG_ID</code>	ID организации. Если не задан, берется первая организация из <code>/orgs/my</code> .
<code>KONTUR_KEDO_EMPLOYEE_ID</code>	ID сотрудника-получателя. Если не задан, берется текущий сотрудник пользователя.
<code>KONTUR_KEDO_DOCUMENT_TYPE_ID</code>	Тип документа по умолчанию.

Переменная	Назначение
KONTUR_KEDO_DOCUMENT_TYPE_NAME	Имя типа документа для авто-выбора, если ID не задан.
KONTUR_KEDO_SIGNATURE_TYPES	Список типов подписи через запятую. По умолчанию Пер,Нер .
KONTUR_KEDO_TEST_FILENAME	Имя тестового файла по умолчанию.
KONTUR_KEDO_TEST_TEXT	Текст для генерации тестового файла, если файл не передан.

3. Общие правила вызова API КЭДО

3.1. Базовый URL

В коде используется KONTUR_KEDO_BASE_URL . Если он уже заканчивается на /kedo , то при вызове методов путь /kedo/... автоматически преобразуется в /... , чтобы итоговый URL не стал .../kedo/kedo/....

Пример:

```
KONTUR_KEDO_BASE_URL=https://api.testkontur.ru/kedo
Метод в коде: /kedo/api/v1/orgs/{orgId}/document-types
Фактический URL: https://api.testkontur.ru/kedo/api/v1/orgs/{orgId}/document-types
```

3.2. Заголовки

Для JSON-запросов:

```
Accept: application/json
Authorization: Bearer {accessToken}
X-Kontur-ApiKey: {apiKey}
```

Для загрузки файла во временное хранилище дополнительно:

```
Content-Type: application/octet-stream
Content-Disposition: attachment; filename="{fileName}"
```

Для скачивания content:

```
Accept: application/octet-stream
Authorization: Bearer {accessToken}
X-Kontur-ApiKey: {apiKey}
```

4. Аутентификация

Метод КЭДО

```
POST /kedo/api/v1/authenticate
```

Request headers

```
Accept: application/json
Content-Type: application/json
X-Kontur-ApiKey: {apiKey}
```

Request body

```
{
  "requestType": "Login",
  "login": "{login}",
  "password": "{password}"
}
```

Response

Ожидается JSON с полем:

```
{
  "accessToken": "..."
}
```

Использование в приложении

Токен получается при каждом пользовательском действии, если явно не передан `access_token`. Если `accessToken` отсутствует, приложение возвращает ошибку авторизации.

5. Получение организации

Если `KONTUR_KEDO_ORG_ID` не задан, приложение запрашивает список организаций.

Метод КЭДО

```
GET /kedo/api/v1/orgs/my
```

Query parameters

Параметр	Значение
<code>limit</code>	100
<code>offset</code>	0

Response

Ожидается pagged-ответ с `result`, из которого берется первая организация.

Нормализованная модель организации:

```
{
  "id": "uuid",
  "name": "string|null",
  "short_name": "string|null",
  "inn": "string|null",
  "kpp": "string|null"
}
```

6. Получение типов документов КЭДО

Endpoint приложения

```
GET /api/kedo/document-types?offset={offset}
```

Query parameters приложения

Параметр	Тип	По умолчанию	Назначение
offset	integer	0	Смещение страницы типов документов. UI использует шаг 20.
filter	string	null	Поддерживается backend-ом, но текущая форма его не использует.

UI-алгоритм

- Кнопка **Получить** типы документов КЭДО вызывает `GET /api/kedo/document-types?offset=0`.
- Кнопка **Следующие 20** увеличивает offset на 20 и вызывает следующую страницу.

Метод КЭДО

```
GET /kedo/api/v1/orgs/{orgId}/document-types
```

Query parameters КЭДО для обычного режима

Параметр	Значение
limit	20
offset	Значение из query приложения
includeDeleted	false
includeDisabled	false
includeSystems	true

Возвращаемое значение приложения

```
{
  "org_id": "uuid",
  "document_types": [
    {
      "id": "uuid",
      "name": "string|null",
      "is_default": false,
      "is_disabled": false,
      "is_formalized": false,
      "metadata": {}
    }
  ]
}
```

7. Получение сотрудников

Endpoint приложения

```
GET /api/kedo/employees
```

Основной метод КЭДО

```
GET /kedo/api/v1/orgs/{orgId}/employees
```

Query parameters

Параметр	Значение
limit	100
offset	0
includeDeleted	false
includeOccupations	true

Fallback-метод

Если основной метод вернул 404 , приложение вызывает:

```
GET /kedo/api/v1/users/my/employees
```

с теми же query parameters.

Возвращаемое значение приложения

```
{
  "org_id": "uuid",
  "employees": [
    {
      "id": "uuid",
      "org_id": "uuid|null",
      "user_id": "uuid|null",
      "full_name": "string|null",
      "login": "string|null"
    }
  ]
}
```

8. Получение типов подписи

Endpoint приложения

```
GET /api/kedo/signature-types
```

Алгоритм

API КЭДО не вызывается. Значения берутся из переменной:

```
KONTUR_KEDO_SIGNATURE_TYPES=Pep,Nep
```

Возвращаемое значение

```
{
  "signature_types": ["Pep", "Nep"]
}
```

9. Отправка файла на подписание

Endpoint приложения

```
POST /api/kedo/test-document
```

Request body приложения

```
{
  "document_type_id": "uuid|null",
  "sender_id": "uuid|null",
  "employee_id": "uuid|null",
  "signature_type": "Pep",
  "due_days": 1,
  "file_name": "test.pdf",
  "file_content_base64": "base64|null"
}
```

Алгоритм

1. Получить `accessToken`.
2. Определить `orgId`.
3. Определить получателя подписания:
 - из `employee_id`;
 - затем из `KONTUR_KEDO_EMPLOYEE_ID`;
 - затем текущий сотрудник пользователя.
4. Определить отправителя:
 - из `sender_id`;
 - если не задан, используется тот же сотрудник, что и получатель.
5. Определить `documentTypeId`:
 - из `request body`;
 - затем из `KONTUR_KEDO_DOCUMENT_TYPE_ID`;
 - затем авто-выбор типа документа.
6. Загрузить файл во временное хранилище.
7. Обработать/конвертировать `content` под тип документа.
8. Создать процесс с документом и маршрутом.
9. Запросить созданный процесс и проверить, что документ создан и не `draft`.
10. Вернуть данные созданного процесса.

9.1. Загрузка файла во временное хранилище

```
POST /kedo/api/v1/orgs/{orgId}/contents
```

Headers:

```
Content-Type: application/octet-stream
Content-Disposition: attachment; filename="{fileName}"
```

Body: бинарное содержимое файла.

Ожидаемый response:

```
{
  "location": "uuid",
  "userId": "uuid|null",
  "name": "file.pdf"
}
```

Критичное поле: `location`.

9.2. Обработка content под тип документа

Старт задачи:

```
POST /kedo/api/v1/orgs/{orgId}/documents/process/tasks
```

Request body:

```
{
  "content": {
    "location": "uuid",
    "userId": null,
    "name": "file.pdf"
  },
  "documentTypeId": "uuid"
}
```

Далее приложение опрашивает задачу:

```
GET /kedo/api/v1/orgs/{orgId}/documents/process/tasks/{taskId}
```

Условия завершения:

- `status = Complete`: берется `result.content` или сам `result`, если там есть `location`;
- `status = Failed` или `Unknown`: ошибка обработки;
- `timeout` обработки: 60 секунд.

Ожидаемый обработанный content:

```
{
  "location": "uuid",
  "userId": "uuid|null",
  "name": "file.pdf"
}
```

9.3. Создание процесса

```
POST /kedo/api/v1/orgs/{orgId}/processes?flat=false
```

Request body:


```

{
  "processes": [
    {
      "name": "KEDO test document 2026-05-18 12:00:00 UTC",
      "documents": {
        "0": {
          "typeId": "documentTypeId",
          "content": {
            "location": "uuid",
            "userId": null,
            "name": "file.pdf"
          }
        }
      }
    },
    "route": {
      "type": "NoAction",
      "id": "uuid",
      "target": {
        "type": "Employee",
        "id": "senderEmployeeId"
      },
      "next": {
        "type": "Sign",
        "id": "uuid",
        "target": {
          "type": "Employee",
          "id": "signerEmployeeId"
        },
        "documentKeys": [0],
        "allowedTypes": ["Pep"],
        "allowedActions": ["Admission", "Rejection"],
        "deadline": {
          "deadlineAt": null,
          "relativeDeadlineAt": 1
        },
        "deadlineAt": null,
        "next": null,
        "comment": null
      },
      "comment": null
    }
  ]
}

```

Важно:

- первая нода маршрута создается как `NoAction`, потому что КЭДО считает первую ноду нодой отправителя;
- действие подписания находится во второй ноде `Sign`;
- `documentKeys` содержит ключ `0`;
- `allowedTypes` заполняется выбранным типом подписи;
- `allowedActions` сейчас `Admission` и `Rejection`;
- `relativeDeadlineAt` равен сроку выполнения в календарных днях.

9.4. Получение созданного процесса

```
GET /kedo/api/v1/orgs/{orgId}/processes/{processId}?flat=false&includeCandidateTargets=false
```

Используется для проверки:

- в процессе есть документы;
- документы не находятся в draft.

Возвращаемое значение приложения

```
{
  "org_id": "uuid",
  "employee_id": "uuid",
  "document_type_id": "uuid",
  "file_name": "file.pdf",
  "content_location": "uuid|null",
  "processed_content_location": "uuid|null",
  "process_ids": ["uuid"],
  "document_ids": ["uuid"],
  "raw_response": [],
  "request_payload": {},
  "process_details": [],
  "download_checks": []
}
```

10. Проверка подписанных документов

Endpoint приложения

```
GET /api/kedo/signed-documents?limit=50&days=14&offset={offset}
```

Query parameters приложения

Параметр	Тип	По умолчанию	Назначение
limit	integer	100 в backend, 50 на кнопке UI	Количество процессов в первичном поиске. Максимум 100.
days	integer	14	Диапазон поиска назад от текущего UTC-времени.
offset	string/null	null	Смещение для /processes/query ; преобразуется в integer.

Алгоритм

1. Получить accessToken .
2. Определить orgId .
3. Найти процессы со статусом Signed за период.
4. Получить детали каждого процесса в плоской модели.

5. Извлечь подписи из `flatPath.nodes[].signedContents`.
6. Вернуть только элементы, где есть `signature.signature.createdAt`.
7. Отфильтровать подписи по диапазону `days`.
8. Отсортировать по `signed_at` по убыванию.

Поиск процессов

```
POST /kedo/api/v1/orgs/{orgId}/processes/query?limit={limit}&offset={offset}
```

Request body:

```
{
  "timeRange": {
    "from": "2026-05-04T00:00:00Z",
    "to": "2026-05-18T00:00:00Z"
  },
  "detailedStatuses": ["Signed"]
}
```

Получение деталей процесса

```
GET /kedo/api/v1/orgs/{orgId}/processes/{processId}?flat=true&includeCandidateTargets=false
```

Детали процессов загружаются параллельно:

- максимум потоков: 8;
- timeout каждого запроса: 10 секунд;
- если отдельный процесс упал по network timeout, он пропускается, остальные процессы обрабатываются.

Извлечение подписи

Приложение ищет ноды типа `Sign`:

- сначала в `flatPath.nodes`;
- затем в рекурсивном `path.firstNode`, если он есть.

Из каждой ноды берется:

```
{
  "signedContents": {
    "0": {
      "signature": {
        "isChecked": true,
        "isValid": true,
        "signature": {
          "id": "uuid",
          "action": "Admission",
          "location": "uuid",
          "createdAt": "2026-05-18T10:00:00Z",
          "author": {
            "employeeId": "uuid",
            "userId": "uuid"
          }
        }
      }
    }
  }
}
```

Критичное поле для попадания в список: `signature.signature.createdAt`.

Возвращаемое значение приложения

```
{
  "org_id": "uuid",
  "last_offset": "50",
  "signed_documents": [
    {
      "process_id": "uuid",
      "process_name": "string|null",
      "process_created_at": "datetime|null",
      "document_key": 0,
      "document_id": "uuid|null",
      "document_name": "file.pdf",
      "signed_at": "2026-05-18T10:00:00Z",
      "action": "Admission",
      "signature_id": "uuid|null",
      "signature_location": "uuid|null",
      "signer_employee_id": "uuid|null",
      "signer_user_id": "uuid|null",
      "is_checked": true,
      "is_valid": true,
      "event_id": null,
      "event_created_at": null
    }
  ]
}
```

11. Скачивание content

Endpoint приложения

```
GET /api/kedo/contents/{fileId}?filename={fileName}
```

Метод КЭДО

```
GET /kedo/api/v1/orgs/{orgId}/contents/{fileId}
```

Возвращаемое значение приложения

HTTP response с бинарным содержимым и заголовком:

```
Content-Disposition: attachment; filename={fileName}; filename*=UTF-8''{fileName}
```

12. Скачивание печатной формы документа

Endpoint приложения

```
GET /api/kedo/processes/{processId}/documents/{documentId}/print
```

Старт задачи печати

```
POST /kedo/api/v1/orgs/{orgId}/processes/{processId}/documents/{documentId}/print/tasks
```

Получение результата

```
POST /kedo/api/v1/orgs/{orgId}/processes/{processId}/documents/{documentId}/print/tasks/{taskId}
```

Алгоритм

1. Создать задачу печати.
2. Опрашивать задачу до `Complete`.
3. Получить `bytes` в `base64`.
4. Декодировать `base64`.
5. Вернуть PDF.

Ограничения

- `timeout` задачи печати: `60` секунд;
- если `bytes` отсутствует, возвращается ошибка `Download KEDO document print`.

13. Диагностические методы приложения

13.1. Проверка доступности КЭДО

```
GET /api/kedo/connectivity
```

Проверяет:

- DNS;
- TCP;
- TLS;
- время проверки.

Response:

```
{
  "url": "https://api.testkontur.ru/kedo",
  "host": "api.testkontur.ru",
  "port": 443,
  "resolved_addresses": ["46.17.203.148"],
  "tcp_connected": true,
  "tcp_error": null,
  "tls_connected": true,
  "tls_version": "TLSv1.3",
  "tls_error": null,
  "elapsed_ms": 884
}
```

13.2. Проверка временного хранилища

```
POST /api/kedo/storage-test
```

Алгоритм:

1. Загрузить файл во временное хранилище.
2. Попробовать скачать загруженный content.
3. Обработать content под тип документа.
4. Попробовать скачать обработанный content.

Response:

```
{
  "org_id": "uuid",
  "document_type_id": "uuid",
  "file_name": "test.pdf",
  "stages": [
    {
      "stage": "upload",
      "ok": true,
      "status_code": 200,
      "location": "uuid",
      "size": null,
      "content_type": null,
      "sha256": null,
      "message": null,
      "raw_response": {}
    }
  ]
}
```

13.3. Сравнение последних документов

```
GET /api/kedo/recent-documents/compare?limit=2
```

Использует:

```
POST /kedo/api/v1/orgs/{orgId}/processes/query?limit={limit}&offset=0
GET /kedo/api/v1/orgs/{orgId}/processes/{processId}
```

Дополнительно пытается скачать content и print для каждого документа.

14. Ошибки

Ошибки КЭДО нормализуются в HTTP 502 от приложения:

```
{
  "detail": {
    "stage": "Get KEDO document types",
    "kontur_status_code": 0,
    "message": "ReadTimeout: The read operation timed out"
  }
}
```

Где:

- stage - внутренний этап алгоритма;
- kontur_status_code - HTTP-код КЭДО, либо 0 при сетевой ошибке;
- message - текст ответа КЭДО или описание исключения.

Ошибки авторизации возвращаются как HTTP 401.

15. Текущие таймауты и ограничения

Операция	Timeout / ограничение
Аутентификация	30 секунд
Получение типов документов	8 секунд , 1 страница по 20 в обычном режиме
Получение сотрудников	30 секунд
Отправка документа	60 секунд на основной HTTP-клиент
Обработка content	опрос задачи до 60 секунд , интервал 1 секунда
Проверка подписанных документов	10 секунд на поиск и на каждый процесс
Детали подписанных процессов	до 8 параллельных запросов
Печать документа	опрос задачи до 60 секунд , интервал 1 секунда

16. Основные внутренние модели

KedoDocumentType

```
{
  "id": "uuid",
  "name": "string|null",
  "is_default": false,
  "is_disabled": false,
  "is_formalized": false,
  "metadata": {}
}
```

KedoEmployee

```
{
  "id": "uuid",
  "org_id": "uuid|null",
  "user_id": "uuid|null",
  "full_name": "string|null",
  "login": "string|null"
}
```

KedoSignedDocument


```
{
  "process_id": "uuid",
  "process_name": "string|null",
  "process_created_at": "datetime|null",
  "document_key": 0,
  "document_id": "uuid|null",
  "document_name": "string|null",
  "signed_at": "datetime",
  "action": "Admission|Rejection|null",
  "signature_id": "uuid|null",
  "signature_location": "uuid|null",
  "signer_employee_id": "uuid|null",
  "signer_user_id": "uuid|null",
  "is_checked": true,
  "is_valid": true
}
```

17. Рекомендуемый боевой сценарий

1. Настроить `KONTUR_KEDO_BASE_URL`, `KONTUR_KEDO_API_KEY`, логин и пароль.
2. Нажать `Получить сотрудников` и выбрать отправителя/участника.
3. Нажать `Получить типы подписи` и выбрать тип подписи.
4. Нажать `Получить типы документов КЭДО`; при необходимости листать `Следующие 20`.
5. Выбрать `KONTUR_KEDO_DOCUMENT_TYPE_ID`.
6. Выбрать файл.
7. Нажать `Отправить тестовый файл в КЭДО`.
8. После подписания нажать `Проверить подписание документов`.
9. Использовать `signed_at`, `signature_id`, `signature_location`, `document_id` и `process_id` для дальнейшей обработки подписанного документа.